Utilizing a Graph Neural Network (GNN) with Deep Q-Learning to Solve Dots and Boxes

Kevin Chen

May 21, 2026

Yilmaz Period 3

Contents

Abstract

Many people are using popular algorithms such as Alpha Beta and Monte Carlo Tree Search to play common games such as Othello or Tic-Tac-Toe. As such, finding the best method to play these games is always on the forefront. In this paper, we are trying to play Dots and Boxes, a deterministic game that involves players drawing lines on a grid in order to complete boxes. However, the state space of Dots and Boxes increases exponentially, which makes algorithms such as Alpha Beta or Monte Carlo Tree Search have to do significantly more computation. Here, the main objective is to train a model to play the game with high proficiency and suffer less from the increase in board size. To resolve this, we utilized a GNN in combination with Deep Q-Learning to try and reduce the time to find an optimal move. Through experimentation, the model achieved an win rate average of 81% across all experiments, and an average of 68.49% boxes captured, with the best performance against the heuristic model on a 5×5 board. It also achieved a superior time complexity, where increasing the board size had a negligible effect on the time to compute the best move. Additionally, the model's performance is inversely proportional to the board size. Ultimately, this paper demonstrates that combining a GNN with Deep-Q Learning is a greatly viable solution for combinatorial games that can be represented as graphs.

1 Introduction

In this paper, I am trying to create a reinforcement learning model using the Deep Q-Learning algorithm with a graph neural network to solve the game Dots and Boxes. I came up with this problem while sitting in my Advanced Placement English Language and Composition class. As class was ending, I noticed a few peers playing a simple game of Dots and Boxes, and I thought to myself, wouldn't it be interesting if I made some sort of agent to play this game? The game seemed very similar to Othello, which I had previously worked on for the Artificial Intelligence 2 class. As such, I wanted to dive further into the realm of reinforcement learning with deterministic games.

Dots and Boxes is what is known as a deterministic game, where there is no chance element, such as dice in a board game, and there is perfect information, such as chess, where both the opponent and the player both know everything there is to the physical game, which is where the pieces are and where they can move. Another property is the termination of the game in a finite set of moves, which allows the existence for a winning strategy. The game is played on a $(n+1) \times (m+1)$ grid of dots. Each player takes turns drawing lines that connect one dot to another adjacent dot, but not diagonally. Eventually, a player can create a box by creating a square with lines, which grants them another turn. The game ends when there are no more empty lines to draw. This results in an endgame state of a $n \times m$ grid of boxes. Here is a labeled game with the vocabulary that is going to be used throughout this paper.

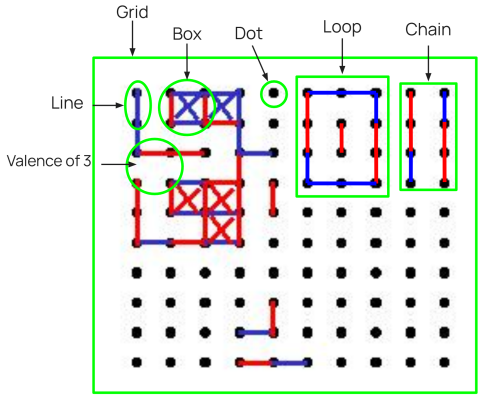

Figure 1: *Labeled game of Dots and Boxes*

Terms

- Dot - A point on the grid
- Line - A connection between two dots
- Grid - The playing zone
- Valence - How many empty lines a box has
- Box - The area enclosed by four lines
- Chain - A sequence of boxes of valence two where two boxes share an empty line
- Loop - A chain where the outermost empty line is shared

At first, the game looks very simple. However, the problem that arises when you try to create a model to play this game is that it has a large amount of games states and it increases exponentially each time. The number of games states can be represented as $g(m, n) = 2^{2mn+n+m}$ where n is the length and m is the width of the board in terms of boxes. As such, when standard search algorithms such as Alpha Beta Pruning or Monte Carlo Tree Search are employed, it takes a long time to search, and becomes inefficient as the board size increases, which is usually how Dots and Boxes played in real life so that the game becomes more interesting, rather than having a short and boring game.

My proposed solution improves upon the implementation and the performance of current models. We are using a GNN over a CNN because a CNN looks for shapes whereas a GNN follows connections and relationships, which allows it to sort of visualize chains and loops and detect optimal moves. Because GNNs look for connections, it is permutation invariant, so the same state that is rotated 90 degrees looks the same to a GNN, but to a CNN it looks completely different. Lastly you can scale the GNN to play any size board because of how the math works. Due to the fact the matrices are inputs to the GNN, you can make the size whatever you want, meaning any size board is playable for the GNN. Compared to regular Q-Learning, the computation of q-values for Deep Q-Learning is more efficient and should improve the performance of the model as well as the computational load that comes with large grids. Through training the GNN, it leverages the natural graph-like structure of the grid of Dots and Boxes to hopefully generalization. I attached a YouTube video which has a small demo of the project [8].

2 Background

A graph is a set of vertices called nodes that are connected by edges. Graphs are occasionally represented as an adjacency matrix to show connections between nodes. The fundamental idea for GNNs is to find a representation for graph data that is suitable for neural networks. This is the representation I will be using, where the horizontal lines are indexed first followed by the vertical lines from the left the right. The boxes are indexed like a matrix. This method was also used in Pandey et. al.'s research. The nodes of the graph represent possible moves (lines) and the edges connect nodes that share the same boxes. Message passing in GNNs is how nodes update by aggregating and transforming their representation based on their neighboring nodes, which is synonymous with convolutional neural networks (CNNs) and how they update each pixel based on their neighboring pixels. This is how nodes learn node features and graph structures. The amount of times you pass a message represents the number of "layers" in a GNN. Learn the embeddings by learning the nodes in your neighborhood. This depends on the size of the graph. Smaller graph means less time to learn about every node, vice versa. Too much message passing leads to oversmoothing (makes all nodes similar to each other). Graph convolutional networks (GCNs) combine aggregating and updating into one computation. Another variant uses Multi Layer Perception as an aggregator making weights learnable. Graph attention networks the importance of the features of the neighbor states are considered for aggregation. All other variants change how aggregation

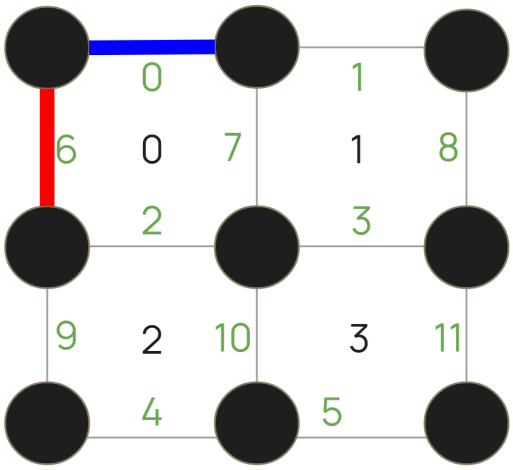

Figure 2: *Board-index representation*

and updating is done. In this paper we will be using the GCN structure for our GNN. In my research, I followed a Keras GNN tutorial to make my custom GNN layer [5].

Reinforcement learning is a machine learning approach where an AI model learns to make decisions by interacting with its environment. For reinforcement learning each (state, action) pair is assigned a q-value, a value that quantifies the future expected reward given the action that is going to be taken. This can be represented in a table where the state and action pairs are listed. However, as the size of the board grows larger, the state-action pairs increase as well and constantly updating these values can get computationally exhausting. So, we want to use something to approximate q-value function such as a neural network, which is where Deep Q-learning comes in. It uses a deep neural network to approximate the q-value for each state action pair in an environment and then the network will approximate the optimal q function. This is especially helpful in a large environment like Dots and Boxes because it saves the program on computational load and time which will most likely produce a faster and smarter model. However, due to the nature of the Dots and Boxes structure being able to be represented as a graph, we can also use GNN in place of a deep neural network. This way, the neural network is able to leverage the graph structure of Dots and Boxes in order to improve generalization. The Deep Q-Learning algorithm pseudo-code I referenced was from the Hugging Face website [3].

This algorithm is the process to train a GNN to learn how to map state-action pairs to q-values, otherwise known as the benefit of playing a certain move. It first starts by initializing a data structure to store the tuples of state, action, reward, and result state tuples later to used to train the generate q-values as well as to train the GNN. Then it creates two GNNs. The action-value GNN to choose the moves during the simulation process, and the target action-value to learn q-values that updates the former GNN's weights to improve its move choosing performance. Then for M episodes, the algorithm plays M games using the epsilon greedy strategy. The initial value for ε is 1. Using the random library from python, if the random value between 0 and 1 is less than epsilon, then it plays a random move. This process is called exploration because the model is trying something new to see the result. Otherwise, if the value is larger, It utilizes the action-value GNN to play the best possible move or the move with the highest q-value. This process is called exploitation because it believes this is the best possible move and will act as such in order to get a higher reward. Then the game simulates the action. I also included the opponents move as part of the simulation to prevent the model from making a choice purely based off the current board state, and instead it has to also if it blunders. Then it stores this interaction in the replay memory. Then if D has enough elements to create one minibatch, number of tuples defined by the user, it samples a random minibatch from D which then find the target q-values for the elements in the

Algorithm 1 Deep Q-Learning Algorithm[3]

Initialize replay memory D to capacity N
Initialize action-value function Q with weights θ
Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$
for episode = 1, M **do**
    Initialize initial game state $s_1 = \{x_1\}$
    for $t = 1, T$ **do**
        With probability ε select a random action a_t
        otherwise select $a_t = \text{argmax}_a Q(\phi(s_t, a; \theta))$
        Execute action a_t in emulator and observe reward
        Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
        Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D
        Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

$$\text{Set } y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases} \tag{1}$$

        Perform a gradient decent step on $(y_j - Q(\phi_{j+1}, a; \theta^-))^2$ with respect to the network parameters θ
    end for
    Every C episodes reset $\hat{Q} = Q$
end for

minibatch where if the state terminates in one step, it just uses the reward from the tuple, otherwise it adds the correction term from the Bellman equation. Then the GNN performs a gradient descent step on the squared difference with respect to the weights in Q. Then, every C steps, The target-action model is set to the action-value GNN. At the end, the action-value model is returned and used as the model to play in games by predicting the move based off the game state.

3 Related Works

In this section, I will introduce a few models that were created to solve the game of Dots and Boxes.

3.1 Rule-Based (Heuristics)[1]

This model created by Simon implements strategic rules, otherwise known as heuristics, that the model must follow while it plays. An overarching rule they include is to ignore all non-empty lines because you can't play on them. In the first phase, which they define as the phase where all boxes have a valence of 1 or more, the model will follow these rules: never create a box with a valence of 1, and be there first player to enter phase 2. Phase 2 is where the boxes all have a valence of 1. As such, the rules defined in this phase are to complete the chains in the order of their lengths and apply the double dealing strategy if beneficial. This algorithm is what is known as greedy, which means the model is always looking to benefit itself. In actual game play, it is sometimes beneficial to sacrifice some points in order to win in the long run, which this model struggles with. As such, greedy algorithms usually lead to huge losses when played against a more sophisticated model.

3.2 Monte Carlo Tree Search[1]

Monte Carlo Tree Search (MCTS) is a best-first search algorithm that explores the game tree by following a policy based on the statistics of certain game states. There are

four main parts to the the algorithm. Selection of the child node is based on the Upper Confidence Bound: $x \in argmax_{i \in I}(v_i + C \cdot \sqrt{\frac{ln_p}{n_i}})$, which gives a score to each node in the tree based on the proportion of simulations based on this node that has led to wins, the number of visits to the node, and the number of visits to the parent node. Next, the children of the selected node are added to the game tree. Then, a simulation is played on the chosen node, where the results of the simulation are used to update the selected node and its parents. The main issue with this method is the increase in computation as the tree increases in size.

3.3 Alpha Beta Pruning[1]

Alpha Beta Pruning is a spin on the MiniMax algorithm, a recursive algorithm used in game theory and artificial intelligence to make decisions assuming that the opponent is also playing optimally. This algorithm aims to prune the tree, in other words, to reduce the number of nodes that the algorithm needs to evaluate. It does this by keeping track of two values, alpha α and beta β which represents the minimum and maximum values, respectively. If the value of the node is less than alpha or greater than beta, it is pruned. The limitations of this method is that its still very expensive for large trees and works best when the best moves are evaluated first.

3.4 Q-Learning with Convolutional Neural Network (CNN)[2]

Q-Learning, as described in the background, gets very expensive as the board size increases. This model utilize a CNN to predict Q-values. During the training process, it utilizes the epsilon greedy strategy, where each epoch has a value ϵ, which is the probability of the agent making a random move, and it slowly decreases each epoch until a certain specified threshold. These games that the epsilon greedy model plays is used as a training set for the neural network in mini batches. Through Pandey's testing, he noticed that the model failed to pick up on important strategies such as double trapping and double crossing.

4 Applications

The applications of this project are limitless. The technique of using Deep Q-Learning with Graph Neural networks is not novel, however, the implementation of it to solve certain puzzles or games is not as prolific. Games that also contain graph structures such as Connect 4, Chinese Checkers, or other games that might seem like they have graph like connections. Extending to the real world, this project can show that agents in graph-like environments, such as rescue efforts in a city full of blocks, could be of significant value by taking the amount of people and the severity of injuries. Additionally, this project can also be used as a test case in future Dots and Boxes projects.

5 Methods

The environment and models were built using Python 3.11, TensorFlow 2.16, and Keras 3. Since the dots and the boxes are constant in every state, the game state can be represented by the edges alone. The game state will then be represented in an adjacency matrix and a node feature matrix. The adjacency matrix represents the connections between the nodes and the node feature matrix represents the state of the board, whether lines are drawn or not and if the lines are on the perimeter of the board.

These pieces of information are passed into the the GNN and the message passing, otherwise known as the calculation between different layers of the neural network, which is calculated as $\mathrm{H}_{i+1} = \sigma(\mathrm{A} \times \mathrm{N} \times \mathrm{W})$, where σ is the activation function, Rectified Linear

Unit, defined as

$$y = \begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases} \tag{2}$$

A, the adjacency matrix, X, the node feature matrix, and W, an arbitrary weight matrix. The H matrix transformed into a different dimension based off the weight matrix and the final H is a vector whose indices map a line to a q-value. Here, we will perform an example calculation and explain the significance of its outcome based on the game in Figure 1. The rows and columns of A represent the indices of the lines where a 1 indicates those two lines share a box and a 0 the opposite. The first column of X indicate whether a line is drawn, 1 meaning it is, 0 meaning it isn't. This resulting matrix is then used in replacement of the node feature matrix the following message passing sequence.

$$\mathrm{A} \times \mathrm{N} = \begin{bmatrix} 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \\ 0 & 1 \\ 0 & 1 \\ 1 & 1 \\ 0 & 0 \\ 0 & 1 \\ 0 & 1 \\ 0 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 2 \\ 0 & 2 \\ 2 & 5 \\ 0 & 5 \\ 0 & 3 \\ 0 & 3 \\ 2 & 2 \\ 2 & 4 \\ 0 & 2 \\ 0 & 3 \\ 0 & 5 \\ 0 & 5 \end{bmatrix}$$

The weight and the activation function in the calculation was omitted in order to demonstrate the main part of this is to create a sum-based aggregation of the node information. The first column of the matrix represents how many edges are played in a shared box while the second column represents the how many edges are on the outside of that line's box and its neighboring boxes.

In gameplay, the model will want to play the move with the highest q-value in order to win. This is optimized through the Bellman equation, which is the second part of the piecewise function in equation 2. The reward calculation utilized in the algorithm is $+1$ for each box captured and -1 for each box the opponent captures. Additionally, I added a -10 just in case it chooses an illegal move.

$$r = \sum \begin{cases} 1 & \text{if model gains a box} \\ -1 & \text{if opponent gains a box} \\ -10 & \text{if illegal move} \end{cases} \tag{3}$$

In this research paper, I am utilizing three specific metrics. The first is the win rate which is calculated by the number of games a model won divided by the number of games played in total, denoted by n, times 100, where G_w denotes the number of games won.

$$\text{Win rate} = \frac{\mathrm{G}_w}{n} \times 100 \tag{4}$$

The next statistic is the percent of boxes captured, which is the number of boxes captured over an n number of games over the total number of boxes in a n number of games times 100 where B is the number of boxes a player captured.

$$\%\text{ of boxes captured} = \frac{\sum^{n} B_{player}}{\sum^{n} (B_{player} + B_{opponent})} \times 100 \tag{5}$$

The last statistic utilized is the average time per move, which is calculated by the total time it took for the player to move in n amount of games divided by the number of games n.

$$\text{Average time per move} = \frac{\sum^{n} t_{player}}{n} \tag{6}$$

I chose these three statistics in order to demonstrate my model's difference in skill. A consistent time to pick its most optimal move along with a sufficiently high win rate means a better model. Additionally, with the boxes captured, it shows how much the model truly dominates microscopically.

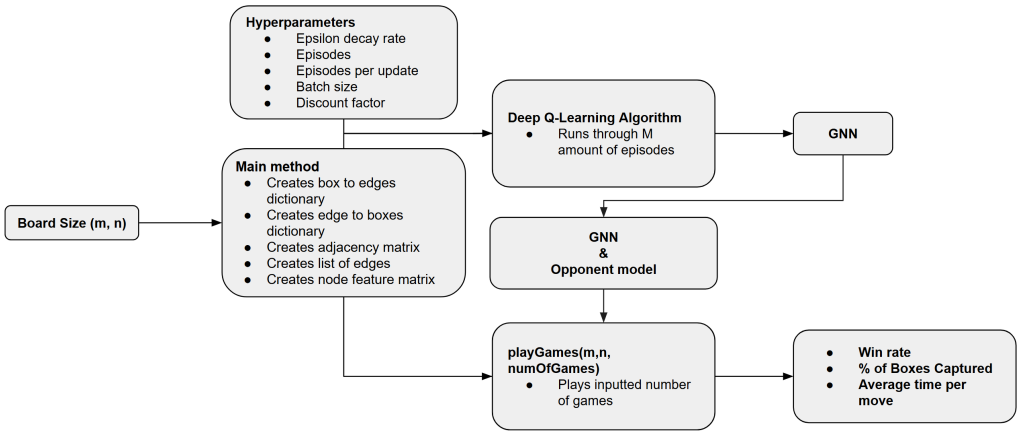

Figure 3: *Program outline*

These previous components were then utilizes in a single program, whose structure is outlined in Figure 2. The user first inputs the size of the grid by boxes, then the main method creates the necessary data structures in order to (i) play games or (ii) train a GNN. Then by commenting out the option you would not like to do, you can train the model by changing the hyperparameters of the Deep Q-Learning algorithm, which outputs a GNN, or you can play any amount of games, which at the end prints out the game statistics outlined in the section above.

The GNN is a 2-layer spatial graph convolutional network, using the Adam Optimizer from Keras with a learning rate of 0.001, and uses Mean Squared Error. I chose two layers in hopes of the model to learn the double dealing strategy, which is letting the opponent fill in the last move of a chain or loop in order for them to start filling in a longer loop or chain, which ends with a net gain for the player, which requires the model to see at lease two moves ahead. One big problem I faced during the implementation of my custom layer, was a deserialization error. I fixed this by referring back to the presentation and adding `**kwargs` and a custom `get_config()` method. For my model, it was trained on 400 episodes with a memory capacity of 2000. Since this is a reinforcement learning environment, data is generated on the fly by the epsilon greedy self play loop. A few modifications were attempted in order to make the model more powerful. In the training process, it was played against itself, which also included an epsilon-greedy strategy. This improved the overall win rate to 50 percent against the Alpha Beta model with a depth of 3, which is the depth used throughout the whole paper. Consequentially, I hypothesized that removing the epsilon greedy strategy would make the model more intelligent because it's opponent was only playing the most optimal move. This turned out to be another successful modification because it had about a win rate of 70-80% against the Alpha Beta model.

Lastly, I performed finetuning of the hyperparameters of the algorithm, which include the epsilon decay factor, the learning rate, the number of episodes, size of the minibatches, and the size of the training boards. The first ever case I used was with an epsilon decay of 0.99, discount factor of 0.95, updates the target action-value GNN one per 10 episodes, and a mini batch of 32. It acheived a 50% win rate against the Alpha Beta Model. I then trained my model for 3500 episodes with an epsilon decay factor of 0.999 on 3x3 boards,

however it only achieved a 26% win rate on Alpha Beta. Increasing training from 400 to 800 episodes only modestly improved performance to 60%. Setting the discount factor to 0.999 caused the win rate to drop from 70% to 60%. I settled on a model with an epsilon decay factor of 0.99, discount rate of 0.99, 400 episodes, minibatch of size 32, and it was trained on 4×4 boards.

Algorithm 2 Heuristic Model

```
for every line in the grid do
    for every box the lines shares do
        if that box has a valence > 1 then
            mark as safe
        end if
    end for
end for
if there are safe moves then
    Play a random safe move
else
    Play any available move
end if
```

For experimentation, the model was played against three other models on three different models: Alpha Beta, Random, and Heuristics. The random model just played a random move chosen by the random library in Python [7]. The Heuristic model is a basic model that performs differently based on the stage it is presently in. Phase 1 is when all the boxes have a valence greater than 1, so it avoids making boxes with a valence of one. After all the boxes have a valence of one, it chooses a random move because it has to make a box regardless of its move. The Alpha Beta model was created based off the code from GeeksToGeeks [4]. The board evaluation that I made is $+1$ for each of the players own boxes, and -1 for each of the opponent's boxes, as written in the following equation, where b is the value of the board and s is the board state.

$$b(s) = \begin{cases} 1 & \text{if own box} \\ -1 & \text{otherwise} \end{cases} \tag{7}$$

6 Results

The model played 100 games on different board sizes and different models as depicted in the first column of Table 1. My model successfully attained a win rate average of 81% across all experiments, which met the objective of the research project. Its best performance can be seen against the heuristic model with a win rate of 94% and captured 77.52% of the boxes. It also had an average of 68.51% boxes captured across all experiments. This means even though my model didn't win as much as I wanted to in some trials, it still successfully scored more points compared to the opponent. It can be observed that the performance of the Deep Q-Learning model with a GNN is inversely related to the size of the board. In all trials, the model successfully attained a box capture rate of over 50% meaning it is superior than the other models in securing points. Additionally, the time per move to the GNN is observed to be relatively the same, except for the last two trials against the random agent, demonstrating that increasing the state size provides negligible resistance to compute the move.

7 Limitations

A big limitation was the hardware. For 400 episodes, it took my laptop about 30-40 minutes to run. This was only on 4×4 boards. The specifications on the device are AMD Ryzen 7 7735HS with Radeon Graphics (3.20 GHz), 16GB Ram, and a NVIDIA

Opponent	Win Rate	Boxes Captured	Time per move for GNN (s)	Time per move for opponent (s)
Alpha Beta of depth 3 (3×3)	71%	60.7%	0.0563	0.0218
Alpha Beta of depth 3 (4×4)	86%	72.56%	0.0544	0.0925
Alpha Beta of depth 3 (5×5)	88%	73.20%	0.0569	6.866
Random (3×3)	72%	59.22%	0.0567	5.62e-6
Random (4×4)	83%	72.31%	0.0785	1.406e-5
Random (5×5)	85%	70.84%	0.0995	8.454e-6
Heuristics (3×3)	71%	59.56%	0.0544	2.177e-5
Heuristics (4×4)	83%	70.69%	0.0538	3.767e-5
Heuristics (5×5)	94%	77.52%	0.0531	5.167e-5

Table 1: *Results of 1 versus 1 of GNN model against other models in 100 games with the win rate, percent of boxes captured, time per move, and time per move for the opponent.*

GeForce RTX3050 6GB Laptop GPU. In real game play boards can get as big as 20×20. Additionally, the boards aren't limited to square boards, they can also be rectangular. That might also affect strategy as well as how loops and chains form. However, this means in testing, models like Alpha Beta will have a long time finding their most optimal move. For my model specifically, the time for training is most likely affected by the slow nature of the Python environment and endless calculations of matrix multiplication. The reason likely for the performance is due to feature over-smoothing. The model was trained on 4×4 boards, it can't really 'see' as well on 3×3 boards and its able to apply its strategies locally on bigger boards. Additionally, after observing one game close up, it makes blunders quite often. When it has the ability to continuously draw lines that complete boxes that result in the termination of the game, it doesn't go for the winning move.

8 Conclusion

This research paper shows that using reinforcement learning, specifically Deep Q-Learning with a GNN, is a strong solution. By representing the game as a graph, it's able to be fed into a GNN which maps each state-action pair to a q-value, and as such simplifies the calculations required in Q-learning through matrix multiplication.

I included a modification to the algorithm by having the opponent play its moves for the calculation of the consecutive state, which significantly helped increase the performance of the model, because it had a good opponent to learn from.

During the testing process, the model showed proficiency over larger boards compared to smaller boards, which demonstrates that the model is able to apply what it learned on smaller boards and apply it locally in larger boards. With an average win rate of 81%, it has more to achieve which is possible with additional episodes of training and more changes to hyperparameter as well as the reward calculation. Additionally, the model shows superiority in the time needed to find its optimal move. This demonstrates that Deep Q-Learning with a GNN is a viable method to solve deterministic games.

9 Future Work

In the future, I intend to tweak the reward calculation a little more. The current calculation doesn't put a lot of emphasis on reducing the number of boxes the opponent captures. By increasing the consequences of the opponent capturing boxes it would increase the number of boxes captured, which in turn might increase the win rate of

the model. It should also solve the issue of blundering significantly. Additionally, I would like to make more models to test my model against. I attempted to make MCTS model using the GeeksToGeeks [6], however it turned out to be unsuccessful because the moves it was making were unintelligible likely due to a bug in the code. Due to the limited time for research and Alpha Beta is already another prominent search algorithm, MCTS was scrapped. Additionally, AlphaZero, is another RL algorithm developed by Google DeepMind, which even beat a world champion in the game Go. I would love to see how my model would put up against one of the best algorithms in the world. Another modification I would make is to add more attributes to the node feature matrix such as whether a certain line is in a loop or chain and the length of that loop or chain. This could possibly help the model be able to identify chains and loops easier and develop the necessary strategy to deal with those structures. Lastly, I would want to experiment with different types of GNNs. The one currently used is a Graph Convolutional Network. Using a Graph Attention Network could add weights to loops and chains which can help the model learn strategies to abuse those structures, which will likely increase the win rate.

References

[1] E. Simon, "ARTIFICIAL, INTELLIGENT AGENTS FOR DOTS AND BOXES," PlayItNice, 2021. https://www.academia.edu/49067480 /ARTIFICIAL_INTELLIGENT_AGENTS_FOR_DOTS_AND_BOXES (accessed Apr. 23, 2026).

[2] A. Pandey, A. Charles, J. Ross, and C. Edwin, "Solving dots & boxes using reinforcement learning," Mountainscholar.org, 2022. https://mountainscholar.org/items/678cab99-8205-46f9-b664-5281ebd6db18 (accessed Apr. 23, 2026).

[3] "The Deep Q-Learning Algorithm - Hugging Face Deep RL Course," huggingface.co. https://huggingface.co/learn/deep-rl-course/unit3/deep-q-algorithm

[4] GeeksforGeeks, "AlphaBeta pruning in Adversarial Search Algorithms," GeeksforGeeks, Jun. 13, 2024. https://www.geeksforgeeks.org/artificial-intelligence/alpha-beta-pruning-in-adversarial-search-algorithms/

[5] K. Team, "Keras documentation: Node Classification with Graph Neural Networks," keras.io. https://keras.io/examples/graph/gnn_citations/

[6] GeeksforGeeks, "Monte Carlo Tree Search (MCTS) in Machine Learning," GeeksforGeeks, Jan. 14, 2019. https://www.geeksforgeeks.org/machine-learning/monte-carlo-tree-search-mcts-in-machine-learning/

[7] Python, "random — Generate pseudo-random numbers — Python 3.8.2 documentation," docs.python.org, 2025. https://docs.python.org/3/library/random.html

[8] K. Chen, "GNN w/ DQL for Dots and Boxes," YouTube, May 16, 2026. [Online]. Available: https://youtu.be/k3YzykCUeFE?si=Stt13mdqd6S8mrO7. [Accessed: May 16, 2026].

A Code

The code, paper, presentation, and model are all incorporated in the following Github repository: https://github.com/jykevon/d-bgnn/tree/main